

DOSSIER D'IMPLÉMENTATION

Multilinguisme

Complet

Français · ■■■■ (RTL) · Türkçe · English

FR

Français

AR



TR

Türkçe

EN

English

Langues cibles : FR (base) · AR RTL · TR · EN — 4 langues complètes**Surfaces** : Laravel (API + Blade) · Flutter (iOS/Android) · Emails · PDFs**RTL** : Architecture Directionality Flutter · Tailwind RTL plugin · HTML dir=rtl**Structure** : 9 chapitres · 55+ tâches · Code de référence complet · Clés de traduction**Périmètre** : UI complet · Messages d'erreur · Emails · PDFs · Formats dates/nombres

4

Langues

2

Surfaces

RTL

Arabe natif

55+

Tâches

TABLE DES MATIÈRES

01	État des lieux & Décisions	Ce qui existe · Ce qui manque · Les 5 décisions architecturales
02	Les 4 langues cibles	FR · AR · TR · EN — profil de chaque langue
03	Architecture i18n — Vue d'ensemble	Le système complet · Flux locale · Source de vérité
04	Laravel — Backend & API	Fichiers lang/ · Middleware locale · Emails · PDFs traduits
05	Flutter — App Mobile	ARB files · intl · Directionality RTL · Formatage
06	Blade — Dashboard Web	Tailwind RTL · Composants · Variables HTML · Emails
07	L'arabe RTL — Guide complet	Règles RTL · Pièges courants · Composants miroir
08	Formats régionaux	Dates · Nombres · Devises · Jours fériés par pays
09	Toutes les tâches	ML-01 → ML-55 · 5 sprints · Priorités · Code de référence
10	Clés de traduction — Inventaire	Toutes les chaînes à traduire · Modules · Contextes
11	Checklist de validation finale	Definition of Done · Tests par langue · RTL validé

CH.01 ÉTAT DES LIEUX & DÉCISIONS

Ce qui existe · Ce qui manque · Décisions architecturales

01

Point de départ honnête : la documentation prévoit 4 langues (FR/AR/TR/EN) et la table languages existe dans l'ERD. En revanche, AUCUNE traduction n'est encore implémentée — pas de fichiers lang/ Laravel, pas de fichiers ARB Flutter, pas de middleware de locale. Ce document est le plan complet pour implémenter tout ça proprement.

Ce qui existe déjà (base à exploiter)

Élément	Emplacement	État	Ce qu'il faut en faire
Table languages	ERD : languages (code, name_fr, name_native, is_rtl, is_active)	Défini dans l'ERD — migration à créer	Créer la migration + seeder avec FR/AR/TR/EN
Champ companies.language	CHAR(2) DEFAULT 'fr' [fr ar tr en]	Dans l'ERD — migration à créer	La locale du manager = langue de toute l'interface
Champ employees.language	Non encore dans l'ERD	À ajouter	Langue préférée de l'employé pour ses notifications
Mention i18n dans PILOTAGE	«i18n préparé (fichiers lang/) mais 1 langue seulement»	Pas encore fait	C'est l'objectif de ce document
Timezone par company	companies.timezone déjà prévu	Dans l'ERD	Déjà en place — les dates/heures suivent la timezone
Currency par company	companies.currency déjà prévu	Dans l'ERD	Déjà en place — les montants suivent la devise
ErrorMessages.dart Flutter	lib/core/api/error_messages.dart	À créer (prévu dans T-DS24)	Base pour les messages d'erreur traduits

Ce qui n'existe pas et doit être créé

À créer	Surface	Criticité	Bloque quoi
Fichiers lang/fr/ lang/ar/ lang/tr/ lang/en/	Laravel	CRITIQUE	Tous les messages API, validation, emails
Middleware SetLocale	Laravel	CRITIQUE	Détection et application de la locale à chaque requête
Fichiers ARB par module Flutter	Flutter	CRITIQUE	Toute l'interface mobile
Directionality RTL dans AppShell Flutter	Flutter	CRITIQUE	Arabe illisible sans RTL
Tailwind RTL plugin (tailwindcss-rtl)	Blade web	CRITIQUE	Dashboard web arabe inutilisable
HTML dir=rtl sur les pages arabes	Blade web	CRITIQUE	Navigateur ne sait pas qu'il faut miroir

À créer	Surface	Criticité	Bloque quoi
Migration languages table + seeder	Laravel	HAUTE	Table référentielle des langues
Champ employees.preferred_language	Laravel	HAUTE	Notifications dans la langue de l'employé
Templates email traduits (4 langues)	Laravel	HAUTE	Emails en arabe et turc
PDF traduits — DomPDF + police arabe	Laravel	HAUTE	Reçus en arabe illisibles sans police
Formatage dates/nombres par locale	Laravel + Flutter	HAUTE	31/12/2026 ≠ 12/31/2026 ≠ ■■■■/■■/■■

Les 5 décisions architecturales — à prendre maintenant

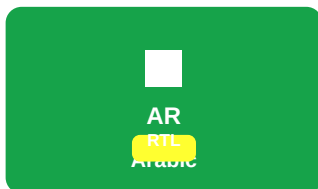
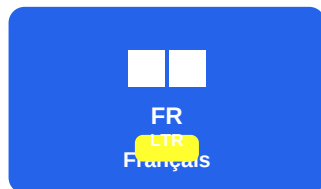
- 1. Source de vérité de la locale :** la locale est déterminée par le champ companies.language pour les managers et par employees.preferred_language pour les employés. Le client Flutter envoie ses préférences mais le serveur est la source de vérité pour les données formatées (emails, PDFs).
- 2. Détection côté serveur :** le middleware SetLocale lit le header Accept-Language de l'API. Si absent, il utilise la locale du compte authentifié (company.language ou employee.preferred_language). Pas de détection automatique par IP.
- 3. Traductions côté client :** Flutter utilise les fichiers ARB (Dart intl). Le serveur Laravel n'injecte PAS les traductions dans les réponses JSON — chaque surface gère ses propres traductions UI. Seuls les données (dates formatées, messages d'erreur) viennent traduits du serveur.
- 4. RTL = inversion complète :** l'arabe RTL n'est pas juste du texte aligné à droite. C'est un miroir complet de l'interface (Directionality.rtl Flutter, dir=rtl HTML, padding/margin inversés). On utilise start/end au lieu de left/right partout.
- 5. Clés de traduction centralisées :** une seule source de vérité pour les clés — le fichier lang/fr/*.php de Laravel ET lib/i10n/app_fr.arb Flutter. Les 3 autres langues sont des traductions de ces fichiers. Jamais de string hardcodée en dehors de ces fichiers.

CH.02

LES 4 LANGUES CIBLES

Profil complet · RTL · Marchés · Spécificités techniques

02



Les 4 langues cibles de Leopardo RH — seul l'arabe est RTL.

Langue	Code	Direction	Marchés visés	Spécificités techniques	Priorité
Français	fr	LTR	Algérie · Maroc · Tunisie · France · Côte d'Ivoire · Sénégal	Langue de base. Tous les fichiers de traduction de référence sont en FR. Aucune spécificité technique particulière.	BASE
Arabe	ar	RTL ← ←	Algérie · Maroc · Tunisie — marché core	RTL obligatoire. Police Noto Naskh Arabic. Chiffres Eastern Arabic optionnels (■ ■ ■ ■ ■). Miroir complet de tous les composants. Tailwind RTL plugin. Flutter Directionality.rtl.	CRITIQUE
Turc	tr	LTR	Turquie — marché secondaire	LTR standard. Attention aux caractères spéciaux : ■/■ (l sans point / i avec point). Collation PostgreSQL 'tr-TR-x-icu' pour le tri correct.	HAUTE
Anglais	en	LTR	International · docs · onboarding	LTR standard. Utilisé pour les intégrations API, la documentation technique, et les clients internationaux.	HAUTE

Seeder de la table languages

LanguagesSeeder.php

// database/seeder/LanguagesSeeder.php

```
Language::upsert([
    ['code'=>'fr','name_fr'=>'Français', 'name_native'=>'Français', 'is_rtl'=>false,'is_active'=>true],
    ['code'=>'ar','name_fr'=>'Arabe', 'name_native'=>'■ ■ ■ ■ ■', 'is_rtl'=>true, 'is_active'=>true],
    ['code'=>'tr','name_fr'=>'Turc', 'name_native'=>'Türkçe', 'is_rtl'=>false,'is_active'=>true],
    ['code'=>'en','name_fr'=>'Anglais', 'name_native'=>'English', 'is_rtl'=>false,'is_active'=>true],
], uniqueBy: ['code']);
```

CH.03

ARCHITECTURE I18N — VUE D'ENSEMBLE

Le système complet · Flux locale · Source de vérité

03

Flux complet de la locale — de la requête à l'affichage

Flux de la locale — de bout en bout

```
# FLUTTER ← La locale vient du profil utilisateur
# 1. L'app lit la locale depuis auth/me.language à la connexion
# 2. Elle initialise MaterialApp avec la locale correspondante
# 3. Directionality.rtl est activé si locale == 'ar'
# 4. Tous les widgets utilisent AppLocalizations.of(context).xxx
# 5. L'app envoie Accept-Language: ar dans chaque requête API

# LARAVEL API ← Le middleware lit et applique la locale
# 1. SetLocale middleware intercepte chaque requête
# 2. Lit Accept-Language header OU employee.preferred_language
# 3. App::setLocale($locale) — valide parmi [fr,ar,tr,en]
# 4. Les messages d'erreur, emails, PDFs sont dans la bonne langue
# 5. Les dates retournées dans l'API sont en ISO 8601 UTC (pas localisées)
# 6. Seules les dates formatées pour affichage (reçus PDF) sont localisées

# BLADE WEB ← La locale vient de la session utilisateur
# 1. Connexion manager → session('locale') = company.language
# 2. app.blade.php ajoute dir='rtl' lang='ar' si locale == 'ar'
# 3. Toutes les vues utilisent __('keys.xxx') pour les chaînes
# 4. Tailwind génère les classes RTL si htmlLang == 'ar'
```

Structure des fichiers de traduction — Laravel

Structure lang/ Laravel

```

api/lang/
■■■ fr/                                # Langue de BASE — source de vérité
■   ■■■ auth.php                      # Messages d'authentification
■   ■■■ attendance.php                # Module Pointage
■   ■■■ finance.php                  # Module Finance
■   ■■■ cameras.php                  # Module Caméras
■   ■■■ employees.php                # Gestion employés
■   ■■■ errors.php                   # Tous les error_codes API
■   ■■■ validation.php               # Messages de validation Laravel
■   ■■■ emails.php                   # Sujets et corps emails
■   ■■■ pdf.php                      # Textes dans les PDFs
■■■ ar/                                # Arabe — traduit depuis fr/
■   ■■■ auth.php                      # ■■■■■■
■   ■■■ attendance.php                # ■■■■■■ ■■■■■■
■   ■■■ ... (même structure)
■■■ tr/                                # Turc
■   ■■■ ... (même structure)
■■■ en/                                # Anglais
    ■■■ ... (même structure)

```

Structure des fichiers de traduction — Flutter

Structure l10n/ Flutter

```

mobile/lib/l10n/                        # Fichiers ARB Flutter (intl)
■■■ app_fr.arb                          # Français — source de vérité
■■■ app_ar.arb                          # Arabe
■■■ app_tr.arb                          # Turc
■■■ app_en.arb                          # Anglais

# Chaque package module a ses propres ARB :
packages/module_rh/lib/l10n/            # Traductions RH
■■■ module_rh_fr.arb
■■■ module_rh_ar.arb
■■■ ...
packages/module_finance/lib/l10n/       # Traductions Finance
■■■ ...
packages/module_cameras/lib/l10n/       # Traductions Caméras
■■■ ...

```

CH.04 LARAVEL — BACKEND & API

Middleware · Fichiers lang/ · Emails · PDFs traduits

04

Middleware SetLocale — le cœur du système

SetLocale.php — middleware de détection de locale

```
// app/Http/Middleware/SetLocale.php
class SetLocale {
    private const SUPPORTED = ['fr', 'ar', 'tr', 'en'];
    private const DEFAULT   = 'fr';

    public function handle(Request $request, Closure $next): Response {
        $locale = $this->resolveLocale($request);
        App::setLocale($locale);
        Carbon::setLocale($locale); // Pour les dates relatives (Carbon::now()->diffForHumans())
        return $next($request);
    }

    private function resolveLocale(Request $request): string {
        // 1. Utilisateur authentifié → sa langue en priorité
        if ($user = $request->user()) {
            $userLocale = $user->preferred_language // employé
                ?? $user->company?->language        // ou company
                ?? null;
            if ($userLocale && in_array($userLocale, self::SUPPORTED)) {
                return $userLocale;
            }
        }
        // 2. Header Accept-Language du client (Flutter envoie 'ar' ou 'fr')
        $header = $request->header('Accept-Language', '');
        $lang   = strtolower(substr($header, 0, 2));
        if (in_array($lang, self::SUPPORTED)) return $lang;
        // 3. Fallback
        return self::DEFAULT;
    }
}
```

```
// Enregistrement dans bootstrap/app.php
->withMiddleware(function (Middleware $m) {
    $m->api(prepend: [SetLocale::class]); // Premier middleware de la pile API
});
```

Fichier lang/fr/errors.php — les codes d'erreur traduits

Ce fichier est critique : il traduit tous les `error_code` de l'API en messages lisibles. C'est le seul endroit où les codes bruts deviennent du texte humain.


```

lang/fr/errors.php
// lang/fr/errors.php
return [
    // Auth
    'INVALID_CREDENTIALS' => 'Email ou mot de passe incorrect.',
    'ACCOUNT_SUSPENDED'   => 'Votre compte a été suspendu. Contactez votre responsable.',
    'ACCOUNT_ARCHIVED'    => 'Ce compte est archivé.',
    'TOKEN_EXPIRED'       => 'Votre session a expiré. Veuillez vous reconnecter.',
    'TOO_MANY_ATTEMPTS'   => 'Trop de tentatives. Réessayez dans :minutes minutes.',
    // Pointage
    'ALREADY_CHECKED_IN'  => 'Vous avez déjà pointé votre arrivée aujourd'hui.',
    'MISSING_CHECK_IN'    => 'Pointez d'abord votre arrivée avant de sortir.',
    'ALREADY_CHECKED_OUT' => 'Vous avez déjà pointé votre départ aujourd'hui.',
    // Finance
    'PLAN_CAMERAS_REQUIRED' => 'Votre plan ne inclut pas le module caméras. Passez au plan Business.',
    'MAX_CAMERAS_REACHED'  => 'Limite de :limit caméras atteinte pour votre plan.',
    'INVOICE_ALREADY_SENT' => 'Cette facture a déjà été envoyée et ne peut plus être modifiée.',
    // Général
    'NOT_FOUND'           => 'Ressource introuvable.',
    'FORBIDDEN'           => 'Vous n'avez pas les droits pour cette action.',
    'SERVER_ERROR'        => 'Une erreur est survenue. Veuillez réessayer.',
];

```

```

lang/ar/errors.php
// lang/ar/errors.php – Version arabe
return [
    'INVALID_CREDENTIALS' => 'البريد الإلكتروني أو كلمة المرور غير صحيحة.',
    'ACCOUNT_SUSPENDED'   => 'حسابك قد تم إيقافه. اتصل بمسؤولك.',
    'ACCOUNT_ARCHIVED'    => 'هذا الحساب قد تم أرشفته.',
    'TOKEN_EXPIRED'       => 'جلسة المستخدم قد انتهت. يرجى إعادة الاتصال.',
    'TOO_MANY_ATTEMPTS'   => 'تجاوزت الحد المسموح به من المحاولات. يرجى المحاولة بعد :minutes دقائق.',
    // Pointage
    'ALREADY_CHECKED_IN'  => 'لقد سجلت بالفعل وصولك اليوم.',
    'MISSING_CHECK_IN'    => 'يجب عليك تسجيل الوصول أولاً قبل الخروج.',
    'ALREADY_CHECKED_OUT' => 'لقد سجلت بالفعل مغادرتك اليوم.',
    // Finance
    'PLAN_CAMERAS_REQUIRED' => 'خطةك لا تتضمن وحدة الكاميرات. انتقل إلى خطة Business.',
    'MAX_CAMERAS_REACHED'  => 'تم الوصول إلى الحد الأقصى لعدد الكاميرات لخطةك.',
    'INVOICE_ALREADY_SENT' => 'هذه الفاتورة قد تم إرسالها بالفعل ولا يمكن تعديلها.',
    // Général
    'NOT_FOUND'           => 'المورد غير موجود.',
    'FORBIDDEN'           => 'ليس لديك الصلاحيات لأداء هذا الإجراء.',
    'SERVER_ERROR'        => 'حدث خطأ. يرجى المحاولة مرة أخرى.',
];

```

Utilisation dans les controllers et services

Utilisation de `trans()` dans le code

```
// Dans un FormRequest ou un Service
// ■ CORRECT – utiliser trans() ou __() partout
throw new ApiException(
    errorCode: 'ALREADY_CHECKED_IN',
    message: __('errors.ALREADY_CHECKED_IN'), // Traduit selon App::getLocale()
);

// ■ CORRECT – messages de validation avec paramètres
__('errors.TOO_MANY_ATTEMPTS', ['minutes' => 5])
// Résultat FR : 'Trop de tentatives. Réessayez dans 5 minutes.'
// Résultat AR : 'تعدد المحاولات. يرجى المحاولة بعد 5 دقائق.'

// ■ INTERDIT – chaînes hardcodées dans le code
throw new ApiException(message: 'You have already checked in today.');
```

Emails traduits — Mail Mailable

InvoiceEmail.php – emails multilingues

```
// app/Mail/InvoiceEmail.php
class InvoiceEmail extends Mailable {
    public function envelope(): Envelope {
        return new Envelope(
            subject: __('emails.invoice_subject', [ // Traduit selon locale active
                'number' => $this->invoice->number,
                'company' => $this->invoice->company->name,
            ]),
        );
    }
    public function content(): Content {
        return new Content(
            markdown: 'emails.invoice.' . App::getLocale(), // emails/invoice/fr.blade.php
            // emails/invoice/ar.blade.php (RTL – template séparé)
        );
    }
}

// Avant d'envoyer l'email – setter la locale vers celle du destinataire
App::setLocale($invoice->client_language ?? $invoice->company->language);
Mail::to($invoice->client_email)->send(new InvoiceEmail($invoice));
App::setLocale($originalLocale); // Remettre la locale d'origine après
```

PDFs traduits — DomPDF + police arabe

```
DomPDF + police arabe — configuration complète
// PROBLÈME : DomPDF ne supporte pas l'arabe nativement
// SOLUTION : Ajouter la police Noto Naskh Arabic dans DomPDF

// 1. Télécharger la police
// wget https://fonts.google.com/download?family=Noto+Naskh+Arabic → storage/fonts/

// 2. config/dompdf.php — déclarer la police
'fonts' => [
    'noto_naskh_arabic' => [
        'R' => storage_path('fonts/NotoNaskhArabic-Regular.ttf'),
        'B' => storage_path('fonts/NotoNaskhArabic-Bold.ttf'),
    ],
],

// 3. Template Blade arabe — resources/views/finance/pdf/invoice_ar.blade.php
// <html dir='rtl' lang='ar'>
// <style>
//   body { font-family: noto_naskh_arabic; direction: rtl; text-align: right; }
// </style>

// 4. Dans FinancePdfService.php — choisir le bon template
$template = 'finance.pdf.invoice_' . App::getLocale();
if (!view()->exists($template)) $template = 'finance.pdf.invoice_fr'; // fallback
$pdf = Pdf::loadView($template, compact('invoice'));
```

CH.05

FLUTTER — APP MOBILE

ARB files · intl · Directionality RTL · Formatage

05

Configuration pubspec.yaml

```
pubspec.yaml + l10n.yaml
# pubspec.yaml — ajouter les dépendances i18n
dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter
  intl: ^0.19.0          # Formatage dates, nombres, pluriels

flutter:
  generate: true          # Active la génération automatique depuis les ARB

# l10n.yaml — configuration du générateur
# Créer ce fichier à la racine du projet mobile :
arb-dir: lib/l10n
template-arb-file: app_fr.arb    # Fichier de base = français
output-localization-file: app_localizations.dart
output-class: AppLocalizations
preferred-supported-locales: ['fr', 'ar', 'tr', 'en']
```

Fichier ARB de base — app_fr.arb (extrait)

lib/l10n/app_fr.arb — fichier de base français

```
{
  "@@locale": "fr",
  "@@last_modified": "2026-04-24",

  "appName": "Leopardo RH",
  "@appName": { "description": "Nom de l'application" },

  "btnCheckIn": "Pointer l'arrivée",
  "@btnCheckIn": { "description": "Bouton principal check-in" },

  "btnCheckOut": "Pointer le départ",

  "statusPresent": "Présent",
  "statusAbsent": "Absent",
  "statusLate": "En retard",
  "statusLeave": "Congé",

  "greetingMorning": "Bonjour {name} ■",
  "@greetingMorning": {
    "description": "Salutation matinale personnalisée",
    "placeholders": { "name": { "type": "String" } }
  },

  "invoicesCount": "{count, plural, =0{Aucune facture} =1{1 facture} other{{count} factures}}",
  "@invoicesCount": {
    "placeholders": { "count": { "type": "int" } }
  },

  "errorInvalidCredentials": "Email ou mot de passe incorrect.",
  "errorAlreadyCheckedIn": "Vous avez déjà pointé votre arrivée aujourd'hui.",
  "errorMissingCheckIn": "Pointez d'abord votre arrivée avant de sortir.",
}
```

Fichier ARB arabe — app_ar.arb (extrait)

lib/l10n/app_ar.arb — fichier arabe

```

{
  "@@locale": "ar",

  "appName": "Leopardo RH",
  "btnCheckIn": "تسجيل الدخول",
  "btnCheckOut": "تسجيل الخروج",
  "statusPresent": "حاضر",
  "statusAbsent": "غائب",
  "statusLate": "متأخر",
  "statusLeave": "إعفاء",

  "greetingMorning": "صباح الخير {name}!",

  "invoicesCount": "{count, plural, =0{لا فواتير} =1{فاتورة واحدة} two{فواتير} few{{count} فواتير}}",
  "@invoicesCount": {
    "placeholders": { "count": { "type": "int" } }
  },

  "errorInvalidCredentials": "البيانات غير صحيحة.",
  "errorAlreadyCheckedIn": "لقد سجلت الدخول بالفعل.",
  "errorMissingCheckIn": "لم يتم تسجيل الدخول.",
}

```

Configuration MaterialApp — localisation + RTL

main.dart — MaterialApp avec localisation

```

// lib/main.dart — configuration i18n complète
return MaterialApp.router(
  // Localisation
  localizationsDelegates: [
    AppLocalizations.delegate,
    GlobalMaterialLocalizations.delegate,
    GlobalWidgetsLocalizations.delegate,
    GlobalCupertinoLocalizations.delegate,
  ],
  supportedLocales: AppLocalizations.supportedLocales,
  // La locale vient du AuthBloc — charge depuis auth/me.language
  locale: context.watch<AuthBloc>().state.locale,
  // !! Ne pas utiliser localeResolutionCallback ici — la locale est pilotée par le profil
  routerConfig: AppRouter.router,
  theme: AppTheme.light,
  darkTheme: AppTheme.dark,
);

// AuthBloc — initialisation de la locale
// Après connexion réussie :
final language = authResponse['employee']['language'] // ou
  ?? authResponse['employee']['company']['language']
  ?? 'fr';
emit(state.copyWith(locale: Locale(language)));

```

Directionality RTL Flutter — AppShell

AppShell.dart — Directionality RTL

```
// lib/core/navigation/app_shell.dart
class AppShell extends StatelessWidget {
  @override Widget build(BuildContext context) {
    final locale = Localizations.localeOf(context);
    final isRtl = locale.languageCode == 'ar';

    // Directionality enveloppe TOUT le contenu de l'app
    return Directionality(
      textDirection: isRtl ? TextDirection.rtl : TextDirection.ltr,
      child: Scaffold(
        body: _buildBody(context),
        bottomNavigationBar: _buildBottomNav(context, isRtl),
      ),
    );
  }
}

// ■ CORRECT — utiliser start/end au lieu de left/right
EdgeInsetsDirectional.only(start: 16, end: 8) // ← LTR: left=16 | RTL: right=16
Alignment.centerEnd                        // ← LTR: right | RTL: left
MainAxisAlignment.start                    // ← LTR: left | RTL: right

// ■ INTERDIT en mode multilingue
EdgeInsets.only(left: 16) // ← Ne respecte PAS le RTL
Alignment.centerRight     // ← Fixe, ignorera RTL
```

Utilisation des traductions dans les widgets

Utilisation des traductions dans les widgets Flutter

```
// ■ CORRECT — via BuildContext
Text(context.l10n.btnCheckIn)          // 'Pointer l'arrivée' ou '■■■■■ ■■■■■'
Text(AppLocalizations.of(context)!.statusPresent)

// Extension pour simplifier (à ajouter dans lib/core/extensions/context_ext.dart)
extension ContextLocalization on BuildContext {
  AppLocalizations get l10n => AppLocalizations.of(this)!;
  bool get isRtl => Directionality.of(this) == TextDirection.rtl;
}

// Utilisation simplifiée :
Text(context.l10n.greetingMorning(name: user.firstName))

// Pluriels (important en arabe — 6 formes de pluriel !)
Text(context.l10n.invoicesCount(3))
// FR: '3 factures' | AR: '3 ■■■■■' | TR: '3 fatura'

// ■ INTERDIT — strings hardcodées dans les widgets
Text('Pointer l'arrivée') // Ne sera jamais traduit
```


CH.06

BLADE — DASHBOARD WEB

Tailwind RTL · dir=rtl · Composants · Variables HTML

06

Layout HTML — dir et lang dynamiques

```
layouts/app.blade.php — dir RTL dynamique
{{-- resources/views/layouts/app.blade.php --}}
<?php
    $locale = app()->getLocale();          // 'fr' | 'ar' | 'tr' | 'en'
    $isRtl = in_array($locale, ['ar']); // Seulement l'arabe est RTL
?>
<!DOCTYPE html>
<html lang='{{ $locale }}' dir='{{ $isRtl ? 'rtl' : 'ltr' }}'>
<head>
    <meta charset='UTF-8'>
    <meta name='locale' content='{{ $locale }}'>
    {{-- Tailwind RTL -- les classes rtl: sont actives si dir=rtl sur <html> --}}
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class='bg-slate-50 text-slate-900'>
    {{-- Sidebar : côté gauche LTR, côté droit RTL --}}
    <div class='flex rtl:flex-row-reverse'>
        <x-sidebar />
        <main class='flex-1 p-6'>
            {{ $slot }}
        </main>
    </div>
</body>
```

Tailwind RTL — configuration et classes

```
tailwind.config.js + classes RTL

// tailwind.config.js — RTL natif (Tailwind v3.3+ supporte RTL nativement)
module.exports = {
  content: ['./resources/**/*.blade.php', './resources/**/*.js'],
  // Pas de plugin nécessaire pour Tailwind v3.3+ — RTL via le variant rtl:
  // Le variant rtl: est automatiquement actif si <html dir='rtl'>
};

// Exemples de classes RTL dans les vues Blade :

// Padding : gauche en LTR, droite en RTL
<div class='ps-4'>                                {{-- ps = padding-start --}}
<div class='pe-4'>                                {{-- pe = padding-end --}}
<div class='ms-4'>                                {{-- ms = margin-start --}}

// Texte : gauche en LTR, droite en RTL
<p class='text-start'>...</p>                    {{-- text-left LTR, text-right RTL --}}
<p class='text-end'>...</p>                      {{-- text-right LTR, text-left RTL --}}

// Icône position : inverser en RTL
<div class='flex rtl:flex-row-reverse'>          {{-- Miroir horizontal complet --}}
<svg class='rtl:scale-x-[-1]'>                    {{-- Flèche inversée en RTL --}}

// Sidebar : à gauche LTR, à droite RTL
<aside class='border-e border-slate-200'>         {{-- border-end: right LTR, left RTL --}}
```

Composant x-attendance-badge — multilingue

```
x-attendance-badge — badge multilingue

{{-- resources/views/components/attendance-badge.blade.php --}}
@props(['status'])
@php
$config = [
  'present' => ['label' => __('attendance.status_present'), 'bg' => 'bg-emerald-100', 'text' => 'text-emerald-800'],
  'absent'  => ['label' => __('attendance.status_absent'), 'bg' => 'bg-red-100', 'text' => 'text-red-800'],
  'late'    => ['label' => __('attendance.status_late'), 'bg' => 'bg-amber-100', 'text' => 'text-amber-800'],
  'leave'   => ['label' => __('attendance.status_leave'), 'bg' => 'bg-blue-100', 'text' => 'text-blue-800'],
][ $status ] ?? ['label' => $status, 'bg' => 'bg-gray-100', 'text' => 'text-gray-700'];
@endphp
<span class='inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-semibold'
      {{ $config['bg'] }} {{ $config['text'] }}>
  {{ $config['label'] }} {{-- 'Présent' | '■' | 'Mevcut' | 'Present' --}}
</span>
```

CH.07 L'ARABE RTL — GUIDE COMPLET

Règles absolues · Pièges courants · Composants miroir

07

L'arabe RTL est la feature différenciatrice de Leopardo RH sur le marché MENA. Aucun concurrent direct ne le propose correctement. Bien fait, c'est un avantage compétitif majeur. Mal fait, c'est une catastrophe d'expérience utilisateur qui fait fuir les clients arabophones.

Le diagramme — LTR vs RTL en pratique

À gauche : interface LTR standard. À droite : interface RTL arabe — miroir complet.

Règles absolues du RTL**Règle 1 — Start/End, jamais Left/Right**

```
// Flutter — TOUJOURS start/end
EdgeInsetsDirectional.only(start: 16) // ■
EdgeInsets.only(left: 16)             // ■ Ne respecte pas RTL

// Blade — TOUJOURS les variantes logiques Tailwind
<div class='ps-4 pe-2 ms-2'>          // ■ padding/margin start/end
<div class='pl-4 pr-2 ml-2'>          // ■ left/right physiques

// CSS natif — logical properties
padding-inline-start: 16px;           // ■ Respecte RTL
padding-left: 16px;                   // ■ Fixe, ignore RTL
```

Règle 2 — Flèches et icônes directionnelles doivent être inversées

```
// Flutter — inverser les icônes de navigation
Icon(context.isRtl ? Icons.arrow_back : Icons.arrow_forward),
// OU utiliser le widget Directionality-aware :
BackButton() // Se retourne automatiquement en RTL ■

// Blade/SVG — miroir horizontal sur les flèches
<svg class='w-4 h-4 rtl:scale-x-[-1] '> // Icône chevron-right mirrée en RTL
  <!-- heroicon-o-chevron-right -->
</svg>

// Icônes qui ne doivent PAS être inversées :
// - Logos (Leopardo logo garde son orientation)
// - Icônes non-directionnelles (cloche, caméra, dollar)
// - Icônes téléphone, email (toujours dans leur sens)
```

Règle 3 — Nombres et ponctuation restent LTR dans un contexte arabe

Les nombres (1234), les pourcentages (85%), les montants (DZD 1,500), les heures (08:30), les dates (2026-04-24) restent toujours écrits de gauche à droite même dans une interface RTL. Unicode gère cela automatiquement si le texte est correctement marqué. Ne PAS inverser les chiffres manuellement.

Règle 4 — Les pluriels arabes ont 6 formes (pas 2 comme en français)

```
Pluriels arabes — les 6 formes

// L'arabe a 6 formes de pluriel :
// - zéro (0) : ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ (aucune facture)
// - un (1) : ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ (une facture)
// - deux (2) : ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ (deux factures) ← DUAL
// - peu (3-10): {n} ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ (3 à 10 factures)
// - beaucoup (11-99): {n} ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ (11 à 99 factures)
// - autre (100+): {n} ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ (100+ factures)

// Dans le fichier ARB :
"invoicesCount": "{count, plural, =0{■ ■ ■ ■ ■ ■ ■ ■ ■ ■} =1{■ ■ ■ ■ ■ ■ ■ ■ ■ ■} two{■ ■ ■ ■ ■ ■ ■ ■ ■ ■} few{{count} ■ ■ ■ ■ ■ ■ ■ ■ ■ ■}"

// En PHP (lang/ar/messages.php) — utiliser les quantifieurs ICU :
// Recommandé : utiliser Spatie Laravel Translatable ou ICU Message Format
```

Pièges courants du RTL — à éviter absolument

Piège	Mauvaise approche	Bonne approche
Sidebar côté mauvais	CSS: left: 0; width: 250px	border-e + Tailwind flex rtl:flex-row-reverse
Input texte arabe aligné à gauche	Pas de dir sur le champ	ou dir='rtl' si toujours arabe
Numéros de facture inversés	Afficher INV-2026-0001 à l'envers	Envelopper dans un span LTR : INV-2026-0001
Date lue à l'envers	2026-04-24 lu comme 42-40-6202	Formater avec IntlDateFormat — résultat est rtl-safe

Piège	Mauvaise approche	Bonne approche
Icône flèche retour dans le même sens	Icône pointe à gauche en RTL	BackButton() Flutter ou rtl:scale-x-[-1] Tailwind
Tableau avec colonnes dans le mauvais ordre	th: Nom Statut Date	dir='rtl' sur le tableau → colonnes mirrées automatiquement
Police latine pour le texte arabe	Utiliser Helvetica pour l'arabe	Google Fonts Noto Naskh Arabic obligatoire pour DomPDF/print
Scroll horizontal dans le mauvais sens	ScrollController sans RTL	SingleChildScrollView respecte Directionality automatiquement
Barre de progression inversée	LinearProgressIndicator toujours de gauche	LinearProgressIndicator respecte Directionality ■

CH.08

FORMATS RÉGIONAUX

Dates · Nombres · Devises · Jours fériés

08

Formatage des dates par locale

Une même date s'affiche différemment selon la locale. L'API retourne toujours ISO 8601 UTC — la mise en forme est faite côté client (Flutter) ou lors de la génération de documents (PDFs, emails).

Locale	Format date	Exemple	Format heure	Notes
Français (fr)	DD/MM/YYYY	24/04/2026	HH:mm	Séparateur : /
Arabe (ar)	DD/MM/YYYY	■ ■ / ■ ■ / ■ ■ ■ ■ ou 24/04/2026	HH:mm	Chiffres arabes optionnels — préférer les chiffres latins pour la lisibilité universelle
Turc (tr)	DD.MM.YYYY	24.04.2026	HH:mm	Séparateur : .
Anglais (en)	DD/MM/YYYY ou MM/DD/YYYY	24/04/2026	HH:mm	Leopardo RH utilise le format DD/MM pour cohérence internationale

Formatage des dates — Flutter + PHP

```
// Flutter — formatage avec intl
import 'package:intl/intl.dart';

String formatDate(DateTime dt, String locale) {
  final format = {
    'fr': DateFormat('dd/MM/yyyy', 'fr'),
    'ar': DateFormat('dd/MM/yyyy', 'ar'), // Chiffres latins par défaut
    'tr': DateFormat('dd.MM/yyyy', 'tr'),
    'en': DateFormat('dd/MM/yyyy', 'en'),
  }[locale] ?? DateFormat('dd/MM/yyyy');
  return format.format(dt.toLocal()); // Convertir UTC → heure locale de l'app
}

String formatTime(DateTime dt, String locale) {
  return DateFormat('HH:mm', locale).format(dt.toLocal());
}

// PHP — formatage dans les PDFs et emails
// Dans FinancePdfService.php :
$date = Carbon::parse($invoice->issue_date)->locale(App::getLocale())->isoFormat('L');
// FR: '24/04/2026' | AR: '24/04/2026' | TR: '24.04.2026'
```

Formatage des montants par devise et locale

Pays	Devise	Code	Format	Exemple
Algérie	Dinar Algérien	DZD	1.250,00 DZD	1250 centimes = 12,50 DZD
Maroc	Dirham Marocain	MAD	1 250,00 MAD	—
Tunisie	Dinar Tunisien	TND	1.250,000 TND	3 décimales
France	Euro	EUR	1 250,00 €	—
Turquie	Livre Turque	TRY	1.250,00 ₺	—
Sénégal	Franc CFA	XOF	1.250 FCFA	Pas de décimale

FinanceFormatter.dart — formatage montants

// Flutter — formatage des montants

// lib/core/utils/finance_formatter.dart

```
class FinanceFormatter {
  static String formatAmount(int cents, String currency, String locale) {
    final amount = cents / 100.0;
    final format = NumberFormat.currency(
      locale: locale,
      symbol: _getCurrencySymbol(currency),
      decimalDigits: _getDecimalDigits(currency), // TND=3, XOF=0, autres=2
    );
    return format.format(amount);
  }
  static String _getCurrencySymbol(String code) => {
    'DZD': 'DZD', 'MAD': 'MAD', 'TND': 'TND', 'EUR': '€', 'TRY': '₺', 'XOF': 'FCFA',
  }[code] ?? code;
  static int _getDecimalDigits(String code) => code == 'TND' ? 3 : code == 'XOF' ? 0 : 2;
}
// Usage : FinanceFormatter.formatAmount(125000, 'DZD', 'fr') → '1 250,00 DZD'
```

CH.09

TOUTES LES TÂCHES — ML-01 À ML-55

5 sprints · Priorités · Code de référence complet

09

55 tâches organisées en 5 sprints. Sprint 1 (fondations) et Sprint 2 (Laravel) sont bloquants — rien d'autre ne peut fonctionner sans eux. Les sprints 3-5 peuvent être partiellement parallélisés.

SPRINT 1 — Fondations (Semaine 1)

Migrations · Seeders · Middleware · Structure fichiers · ~4,5 j-dev

- ML-01

Migration table languages
[Laravel](#) Créer la migration pour la table languages (code CHAR(2), name_fr, name_native, is_rtl, is_active). Contrainte UNIQUE sur code. Index sur is_active. 0,25 j · Critique
- ML-02

Migration languages seeder
[Laravel](#) LanguagesSeeder insérer FR, AR (is_rtl=true), TR, EN. Idempotent (upsert). Lancer avec DatabaseSeeder. 0,25 j · Critique
- ML-03

Migration companies language
[Laravel](#) Ajouter le champ language CHAR(2) DEFAULT 'fr' FK → languages.code dans la migration companies (si pas encore présent). 0,25 j · Critique
- ML-04

Migration employees preferred language
[Laravel](#) Ajouter le champ language CHAR(2) NULL FK → languages.code dans la table employees. NULL = utilise la langue de la company. 0,25 j · Haute
- ML-05

Middleware SetLocale
[Laravel](#) Créer app/Http/Middleware/SetLocale.php (voir Ch.04). Enregistrer en premier dans la pile middleware API. Tests : FR par défaut, AR via header, TR via profil employé. 0,75 j · Critique
- ML-06

Structure dossiers lang/
[Laravel](#) Créer lang/fr/ lang/ar/ lang/tr/ lang/en/ avec les fichiers vides : auth.php, attendance.php, finance.php, employees.php, errors.php, validation.php, emails.php, pdf.php. Copier les fichiers vendor/laravel/framework/lang/ pour validation.php. 0,5 j · Critique
- ML-07

Lang/fr/errors.php complet
[Laravel](#) Remplir le fichier errors.php avec tous les codes erreur actuels et à venir (voir Ch.04). C'est la source de vérité — les autres langues sont des traductions de ce fichier. 0,5 j · Critique
- ML-08

Lang/fr/attendance.php
[Laravel](#) Tous les textes du module Pointage : statuts (présent, absent, en retard, congé), boutons, messages, titres écrans. 0,25 j · Critique
- ML-09

Lang/fr/auth.php
[Laravel](#) Messages d'authentification : login, logout, session expirée, compte suspendu, bienvenue. 0,25 j · Critique

● ML-10

Config l10n.yaml Flutter

Créer l10n.yaml à la racine du projet mobile. Configurer arb-dir, template, output-class. Ajouter flutter_localizations dans pubspec.yaml. Vérifier que flutter pub get fonctionne.

0,25 j · Critique

● ML-11

Fichier app_fr.arb de base

Créer app_fr.arb avec toutes les chaînes UI de l'app : login, attendance, navigation, settings, erreurs, notifications. Minimum 80 clés pour couvrir tous les écrans existants.

1 j · Critique

● ML-12

MaterialApp localisation + Directionality

Configurer MaterialApp avec localizationsDelegates, supportedLocales. Modifier AppShell pour envelopper dans Directionality selon la locale. Extension ContextLocalization dans context_ext.dart.

0,5 j · Critique

SPRINT 2 — Traductions FR complètes (Semaine 2)

Tous les fichiers lang/fr/ · Tous les ARB FR · Remplacer les hardcodes · ~8 j-dev

● ML-13

Lang/fr/finance.php

Module Finance : statuts factures (brouillon, envoyée, payée, en retard, annulée), catégories dépenses par défaut, messages erreurs finance, titres sections.

0,5 j · Critique

● ML-14

Lang/fr/employees.php

Module Employés : rôles (manager, employé, gestionnaire), statuts (actif, suspendu, archivé), messages gestion employés.

0,25 j · Haute

● ML-15

Lang/fr/emails.php

Tous les sujets et contenus emails : invoice_subject, welcome_subject, password_reset, morning_brief. Variables paramétrables avec :name, :company, :amount.

0,5 j · Haute

● ML-16

Lang/fr/pdf.php

Tous les textes des PDFs générés : titres, colonnes tableaux, mentions légales, pied de page. Utilise par DomPDF dans les templates Blade.

0,25 j · Haute

● ML-17

Remplacer tous les hardcodes Laravel

grep -r message __app__ pour trouver les strings hardcodées. Remplacer par __('errors.CODE') ou __('module.clé'). 0 string hardcodée tolérée dans les controllers et services.

1 j · Critique

● ML-18

ARB FR — module RH (module_rh)

Créer packages/module_rh/lib/l10n/module_rh_fr.arb. Toutes les chaînes des écrans : AttendanceScreen, HistoryScreen, EmployeeListScreen, EmployeeDetailScreen. Référencer depuis AppShell.

1 j · Critique

● ML-19

ARB FR — module Finance (module_finance)

packages/module_finance/lib/l10n/module_finance_fr.arb. Toutes les chaînes : InvoiceDashboard, InvoiceCreate, ExpenseCreate, TreasuryScreen. Statuts, boutons, labels.

1 j · Critique

● ML-20

ARB FR — module Caméras (module_cameras)

packages/module_cameras/lib/l10n/module_cameras_fr.arb. CameraList, CameraView, accès tiers.

0,5 j · Haute

● ML-21

Remplacer tous les hardcodes Flutter

grep -r context __lib__ pour trouver les strings hardcodées. Remplacer par context.l10n.xxx. 0 string hardcodée tolérée dans les widgets. Script de vérification CI.

1,5 j · Critique

● ML-22

Blade — remplacer les strings hardcodées

Toutes les vues blade : remplacer les textes fixes par __('module.clé'). Vérifier que toutes les vues passent par les fichiers lang/fr/.

1 j · Critique

● ML-23

Tests d'intégrité FR

Flutter test : chaque error_code de l'API a une traduction dans lang/fr/errors.php. Flutter test : toutes les clés ARB sont utilisées dans un widget (pas de clé orpheline).

0.5 j · Haute

SPRINT 3 — Arabe RTL

Traductions AR · RTL Flutter · Tailwind RTL · DomPDF police · ~10 j-dev

(Semaines 3-4)

- ML-24

Lang/ar/errors.php

Laravel

Traduire errors.php en arabe. Tous les codes doivent avoir une traduction. Vérifier avec un locuteur natif si possible, sinon avec DeepL + révision.

0,5 j · Critique
- ML-25

Lang/ar/attendance.php

Laravel

Statuts et messages du pointage en arabe. ■■■■ · ■■■■ · ■■■■■ · ■■■■■■.

0,25 j · Critique
- ML-26

Lang/ar/auth.php + emails.php

Laravel

Messagerie d'auth et emails en arabe. Attention : l'objet de l'email doit aussi être en arabe quand la locale est 'ar'.

0,5 j · Haute
- ML-27

Templates email arabe (RTL)

Laravel

Créer les fichiers views/emails/invoice/ar.blade.php avec dir=rtl, police arabe. Le layout est miroir. Utiliser font-family:Noto Naskh Arabic dans le CSS inline.

1 j · Haute
- ML-28

DomPDF — police Noto Naskh Arabic

Laravel

Tester et configurer la police arabe dans DomPDF (voir Ch.04). Créer les templates PDF arabe (invoice_ar.blade.php, receipt_ar.blade.php). Tester la génération d'un PDF en arabe.

0,75 j · Haute
- ML-29

ARB AR — core + module RH

Flutter

app_ar.arb + module_rh_ar.arb. Traductions complètes de toutes les clés FR. Pluriels arabes (6 formes). Vérification par locuteur natif ou DeepL.

1,5 j · Critique
- ML-30

ARB AR — module Finance + Caméras

Flutter

module_finance_ar.arb + module_cameras_ar.arb. Traductions complètes.

1 j · Haute
- ML-31

Flutter RTL — audit complet Edgelsnsets

Flutter

Parcourir tous les widgets et remplacer Edgelsnsets par EdgelsnsetsDirectional. left/right → start/end. Alignment.centerLeft → Alignment.centerStart. Script d'audit automatisé.

1 j · Critique
- ML-32

Flutter RTL — icônes directionnelles

Flutter

Identifier toutes les icônes directionnelles (flèches retour, chevrons, flèches navigation). Appliquer Transform.scale(scaleX: context.isRtl ? -1 : 1) ou utiliser les icônes RTL-aware.

0,5 j · Haute
- ML-33

Blade RTL — audit complet

Blade

Parcourir toutes les vues Blade. Remplacer pl-/pr-/ml-/mr- par ps-/pe-/ms-/me-. Remplacer text-left/text-right par text-start/text-end. flex-row → rtl:flex-row-reverse si nécessaire.

1 j · Critique
- ML-34

Test visuel complet arabe

Transversal

Tester l'app complète en mode arabe : Flutter simulateur + Blade sur navigateur. Vérifier : sidebar côté droit, textes alignés à droite, flèches inversées, tableaux miroirs, dates lisibles, montants lisibles.

1 j · Critique

SPRINT 4 — Turc & Anglais (Semaine 5)

Traductions TR + EN · Spécificités turques · ~5 j-dev

- **ML - 35** **Lang/tr/ complet**
Faire tous les fichiers lang/fr/ en turc. Attention aux caractères ■/■. Collation base de données : si besoin de tri correct en turc, configurer 'tr-TR-x-icu'. 1 j · Haute
- **ML - 36** **Lang/en/ complet**
Faire tous les fichiers lang/fr/ en anglais. Pluriels simples (1 facture / N factures). Pas de spécificité technique particulière. 0,75 j · Haute
- **ML - 37** **APP TR + EN — core + tous modules**
app-tr-and + app-en-and + tous les modules. Pour TR : remplacer i par ■ en majuscule (problème connu Dart intl). Pour EN : pluriels simples. 2 j · Haute
- **ML - 38** **Test caractères turcs ■/■**
Vérifier que Istanbul.toUpperCase() = ■STANBUL en turc (pas 'ISTANBUL'). Utiliser toUpperCase() avec locale si nécessaire ou gérer manuellement. 0,25 j · Haute
- **ML - 39** **Validation orthographique (tous fichiers)**
Passer chaque fichier de traduction (lang/ et ARB) par un correcteur orthographique. Lister les chaînes à faire valider par un locuteur natif pour AR et TR. 1 j · Haute

SPRINT 5 — Formats régionaux & Finalisation (Semaine 6)

Dates · Montants · Sélecteur langue · CI/CD · ~6 j-dev

- **ML - 40** **FinanceFormatter.dart**
Créer mycoreutils/finance_formatter.dart (voir Ch.08). Format montants selon devise et locale. Tests unitaires : DZD FR, DZD AR, EUR, TRY. 0,5 j · Haute
- **ML - 41** **DateFormatter.dart**
Créer mycoreutils/date_formatter.dart. Format dates selon locale. Utiliser intl DateFormat. Tests : 2026-04-24 → '24/04/2026' (FR/AR/EN) et '24.04.2026' (TR). 0,5 j · Haute
- **ML - 42** **Écran paramètres — sélecteur de langue**
Dans SettingsScreen : liste des 4 langues (avec leur nom natif). Tap → PATCH /api/v1/profile/language → mise à jour locale de l'app immédiate (sans reload). Sauvegarde en local storage + en base. 0,75 j · Haute
- **ML - 43** **Sélecteur langue — Blade web**
Dans le menu profil du dashboard : dropdown de sélection de langue. POST /web/settings/language → session + PATCH profil. Rechargement de la page avec la nouvelle locale. 0,5 j · Haute
- **ML - 44** **PATCH /api/v1/profile/language (endpoint)**
Nouvel endpoint pour changer la langue préférée de l'employé/manager. Met à jour employees.preferred_language. Retourne la nouvelle locale dans la réponse. 0,25 j · Haute
- **ML - 45** **Formatage dates dans les réponses API**
Les réponses API retournent les dates en ISO 8601 UTC. Vérifier qu'aucune date n'est déjà formatée localement dans les serializers. Si trouvé : corriger pour retourner ISO 8601. 0,5 j · Haute
- **ML - 46** **Formatage dates dans les PDFs et emails**
Dans le service et les templates email : utiliser Carbon::parse()->locale(App::getLocale())->isoFormat('L') pour formater les dates dans la langue correcte. 0,5 j · Haute

● ML-47	CI — vérification clés manquantes Script CI Flutter Actions : pour chaque clé dans lang/fr/, vérifier qu'elle existe dans lang/ar/, lang/tr/, lang/en/. Échec CI si une clé manque dans une langue. Même vérification pour les ARB Flutter. Transversal	0,5 j · Haute
● ML-48	CI — vérification zéro-hardcode Script CI Flutter Actions : CF: Text() dans les widgets Flutter → 0 résultat attendu. '__(' absent dans les controllers Laravel → 0 résultat attendu. Transversal	0,5 j · Haute
● ML-49	Test de non-régression — FR (smoke test) Transversal Après toutes les améliorations, rejouer les tests de validation du module Pointage (PT-01 à PT-30) en locale FR. S'assurer que rien n'est cassé.	0,5 j · Critique
● ML-50	Test arabe — smoke test complet Transversal Tester en mode arabe : login, check in, today, historique, daily summary. Vérifier les messages d'erreur en arabe. Vérifier les emails envoyés en arabe. Vérifier un PDF généré en arabe.	1 j · Critique
● ML-51	Test turc — smoke test Transversal Même processus que ML-50 en turc.	0,25 j · Haute
● ML-52	Test anglais — smoke test Transversal Même processus en anglais.	0,25 j · Haute
● ML-53	Documentation MULTILANG.md Transversal Créer docs/MULTILANG.md : règles de traduction, comment ajouter une nouvelle langue, comment tester le RTL, comment vérifier les pluriels. Guide pour les futurs développeurs.	0,5 j · Haute
● ML-54	Mise à jour PILOTAGE.md Transversal Mettre à jour la ligne 'Non préparé mais 1 langue seulement' → 'FR + AR + TR + EN — 4 langues opérationnelles'. Ajouter le multilinguisme dans les features livrées.	0,25 j · Haute
● ML-55	Mettre à jour ErrorMessage.dart Flutter Flutter Mettre à jour lib/core/app/src/_messages.dart pour utiliser AppLocalizations au lieu de strings hardcodées. Tous les messages d'erreur doivent passer par le système i18n.	0,5 j · Critique

Sprint	Thème	Semaine	Tâches	Effort	Résultat
Sprint 1	Fondations	Sem. 1	12	~4,5 j	Infrastructure i18n en place
Sprint 2	Traductions FR	Sem. 2	11	~8 j	App 100% en français, 0 hardcode
Sprint 3	Arabe RTL	Sem. 3-4	11	~10 j	App fonctionnelle en arabe avec RTL
Sprint 4	Turc & Anglais	Sem. 5	5	~5 j	4 langues opérationnelles
Sprint 5	Formats & Finalisation	Sem. 6	16	~7,5 j	CI vérifié, formats locaux, sélecteur langue
TOTAL	Module Multilinguisme complet	6 semaines	55 tâches	~35 j-dev	FR+AR+TR+EN production-ready

CH.10

CLÉS DE TRADUCTION — INVENTAIRE

Toutes les chaînes à traduire · Par module · Contextes

10

Inventaire lang/fr/errors.php — codes API

Ces codes sont retournés par l'API et affichés à l'utilisateur. Chaque code doit avoir une traduction dans les 4 langues.

Code erreur	Message FR	Contexte d'affichage
INVALID_CREDENTIALS	Email ou mot de passe incorrect.	Écran login
ACCOUNT_SUSPENDED	Votre compte a été suspendu. Contactez votre responsable.	Login + actions
ACCOUNT_ARCHIVED	Ce compte est archivé.	Login
TOKEN_EXPIRED	Votre session a expiré. Veuillez vous reconnecter.	Toute l'app
TOO_MANY_ATTEMPTS	Trop de tentatives. Réessayez dans :minutes minutes.	Login
ALREADY_CHECKED_IN	Vous avez déjà pointé votre arrivée aujourd'hui.	Check-in
MISSING_CHECK_IN	Pointez d'abord votre arrivée avant de sortir.	Check-out
ALREADY_CHECKED_OUT	Vous avez déjà pointé votre départ aujourd'hui.	Check-out
PLAN_CAMERAS_REQUIRED	Votre plan ne inclut pas le module caméras.	Module caméras
MAX_CAMERAS_REACHED	Limite de :limit caméras atteinte pour votre plan.	Ajout caméra
PLAN_FINANCE_REQUIRED	Votre plan ne inclut pas le module finance.	Module finance
FINANCE_MAX_DOCS_REACHED	Limite de :limit documents atteinte ce mois.	Création facture
INVOICE_ALREADY_SENT	Cette facture ne peut plus être modifiée (déjà envoyée).	Modification facture
NOT_FOUND	Ressource introuvable.	Générique
FORBIDDEN	Vous n'avez pas les droits pour cette action.	Générique
UNAUTHENTICATED	Connexion requise.	Générique
SERVER_ERROR	Une erreur est survenue. Veuillez réessayer.	Générique

Code erreur	Message FR	Contexte d'affichage
VALIDATION_ERROR	Certains champs sont incorrects.	Formulaires
CAMERA_TOKEN_EXPIRED	L'accès à cette caméra a expiré.	Viewer caméra tiers
CAMERA_TOKEN_REVOKED	Cet accès a été révoqué.	Viewer caméra tiers

Inventaire clés UI Flutter (app_fr.arb)

Clé ARB	Valeur FR	Module	Paramètres
appName	Leopardo RH	Core	—
btnCheckIn	Pointer l'arrivée	RH	—
btnCheckOut	Pointer le départ	RH	—
greetingMorning	Bonjour {name} ■	Home	name: String
statusPresent	Présent	RH	—
statusAbsent	Absent	RH	—
statusLate	En retard	RH	—
statusLeave	Congé	RH	—
invoicesCount	{count} facture(s)	Finance	count: int — pluriel
expensesCount	{count} dépense(s)	Finance	count: int — pluriel
hoursWorked	{hours}h travaillées	RH	hours: String
lateByMinutes	En retard de {minutes} min	RH	minutes: int
checkInTime	Arrivée : {time}	RH	time: String
checkOutTime	Départ : {time}	RH	time: String
noDataToday	Aucun pointage aujourd'hui	RH	—
totalTodayAmount	Total aujourd'hui : {amount}	Finance	amount: String
cameraOffline	Caméra hors ligne	Caméras	—
cameraConnecting	Connexion au flux...	Caméras	—
navRH	RH	Navigatio n	—
navFinance	Finance	Navigatio n	—
navCameras	Caméras	Navigatio n	—

Clé ARB	Valeur FR	Module	Paramètres
navSettings	Paramètres	Navigatio n	—
settingsLanguage	Langue de l'interface	Settings	—
btnSave	Enregistrer	Génériqu e	—
btnCancel	Annuler	Génériqu e	—
btnSend	Envoyer	Génériqu e	—
btnDelete	Supprimer	Génériqu e	—
confirmDelete	Confirmer la suppression ?	Génériqu e	—
errorGeneric	Une erreur est survenue. Réessayez.	Erreurs	—
noInternetConnection	Pas de connexion internet.	Erreurs	—

CH.11

CHECKLIST DE VALIDATION FINALE

Definition of Done · Tests par langue · RTL certifié

11

La feature multilinguisme est **TERMINÉE** quand tous les éléments de cette checklist sont ✓ **VALIDÉS**. Le module est certifié multilingue production-ready quand les 4 blocs sont complets.

Bloc 1 — Infrastructure i18n

☐	Migration table languages appliquée	code · name_native · is_rtl
☐	Seeder FR/AR/TR/EN exécuté	4 langues dans la table languages
☐	Middleware SetLocale actif en premier dans la pile API	Test : header Accept-Language: ar → app locale = ar
☐	companies.language et employees.preferred_language en base	Champs présents avec valeurs par défaut
☐	Structure lang/fr/ lang/ar/ lang/tr/ lang/en/ créée	Tous les fichiers présents et non vides
☐	Structure l10n/ Flutter créée avec app_fr.arb de base	Génération auto flutter pub run intl_utils:generate OK
☐	MaterialApp configuré avec localizationsDelegates	Test : changer locale → interface change

Bloc 2 — Zéro hardcode (critique)

☐	grep -r "Text(" lib/ → 0 résultat (sauf commentaires)	CI automatique
☐	grep -r "'message' =>" app/Http app/Services → 0 résultat sans __()	CI automatique
☐	Toutes les vues Blade utilisent __('module.clé')	Vérification manuelle + grep
☐	ErrorMessages.dart Flutter utilise AppLocalizations	Pas de strings hardcodées
☐	Toutes les clés FR ont une entrée dans AR, TR, EN	Script CI de vérification

Bloc 3 — RTL Arabe certifié (critique)

☐	Sidebar apparaît à DROITE en mode arabe	Test visuel Flutter simulateur et navigateur
☐	Textes alignés à DROITE en mode arabe	Vérifier sur AttendanceScreen, LoginScreen, tous les formulaires
☐	Flèches de navigation inversées en mode arabe	BackButton, chevrons, flèches retour
☐	Tableaux dashboard miroirs en mode arabe	Vérifier l'ordre des colonnes
☐	PDF généré en arabe avec police Noto Naskh Arabic	Tester avec FinancePdfService en locale ar
☐	Email envoyé en arabe avec dir=rtl	Vérifier dans Mailtrap
☐	Pas de texte tronqué côté gauche en RTL	Vérifier que le contenu ne déborde pas

☐	Dates et montants lisibles en arabe (chiffres latins)	Vérifier que 1250 reste 1250, pas ■■■■
☐	Pluriels arabes corrects (6 formes)	Test avec 0, 1, 2, 3, 11, 100 éléments
Bloc 4 — Smoke tests 4 langues		
☐	Login FR → check-in FR → message erreur en FR	Test manuel ou Pest
☐	Login AR → check-in AR → message erreur en ARABE	Test manuel — vérifier RTL activé
☐	Login TR → check-in TR → message erreur en TURC	Test manuel
☐	Login EN → check-in EN → message erreur en ANGLAIS	Test manuel
☐	Sélecteur de langue dans Settings Flutter fonctionne	Changer langue → interface change immédiatement
☐	Sélecteur de langue dans dashboard web fonctionne	Changer langue → page rechargée avec RTL si AR
☐	PDF facture généré en FR, AR, TR, EN	Vérifier 4 PDFs — lisibles et correctement formatés
☐	Email envoyé en FR, AR, TR, EN	Vérifier 4 emails dans Mailtrap

Le multilinguisme de Leopardo RH (FR · AR RTL · TR · EN) est certifié production-ready quand les 4 blocs sont complets. L'arabe RTL est le différenciateur concurrentiel le plus fort sur le marché MENA — investir le temps nécessaire pour le faire correctement. Version 1.0 — Avril 2026.